ENOMY FINANCE JAVA CLASSES DESCRIPTION

1. Config Package

Infrastructure and application initialization:

- **DatabaseConfig**
- **SecurityConfig**
- **SecurityWebApplicationInitializer**
- **WebAppInitializer**
- **WebConfig**

2. Security Package

Authentication and authorization:

- CustomAuthenticationFailureHandler
- CustomAuthenticationSuccessHandler
- CustomUserDetails
- CustomUserDetailsService

3. Model Package

Core entities:

User Management

- User
- LoginActivity

Currency Conversion

- CurrencyTransaction
- ConversionRuleSet
- ConversionFeeRule

Investment

- InvestmentQuote
- PlanRules
- TaxSettings

4. DTO Package

Conversion DTOs

- CurrencyConversionRequestDTO
- CurrencyConversionResponseDTO
- CheckRateResponseDTO
- TransactionReceiptDTO
- ConversionFeeRuleDTO
- ConversionRuleSetDTO
- AdminCurrencyRateRowDTO
- AdminCurrencyTransactionHistoryRowDTO
- CurrencyRateApiDTO

Investment DTOs

- InvestmentRequestDTO
- InvestmentResponseDTO
- YearlyInvestmentResultDTO
- PlanDetailsDTO
- AdminInvestmentQuoteHistoryRowDTO
- TaxSetHistoryDTO

User/Profile DTOs

- ClientProfilePageDTO
- ProfileInfoUpdateDTO
- ProfileAvatarUpdateDTO
- ChangePasswordDTO

- LoginActivityFilterDTO

5. DAO & DAOImpl Package

Conversion DAO

- ConversionFeeRuleDao
- ConversionFeeRuleDaoImpl
- ConversionRuleSetDao
- ConversionRuleSetDaoImpl
- CurrencyTransactionDao
- CurrencyTransactionDaoImpl

Investment DAO

- InvestmentQuoteDao
- InvestmentQuoteDaoImpl
- PlanRulesDao
- PlanRulesDaoImpl
- TaxSettingsDao
- TaxSettingsDaoImpl

User DAO

- LoginActivityDao
- LoginActivityDaoImpl
- UserDao
- UserDaoImpl

6. Service Package

Admin Services

- AdminCurrencyService

- AdminCurrencyServiceImpl
- AdminInvestmentService
- AdminInvestmentServiceImpl
- AdminTransactionHistoryService
- AdminTransactionHistoryServiceImpl

Client Services

- ClientProfileService
- ClientProfileServiceImpl
- CurrencyApiService
- CurrencyApiServiceImpl
- CurrencyConverterService
- CurrencyConverterServiceImpl
- InvestmentService
- InvestmentServiceImpl

7. Controller Package

Admin Controllers

- AdminDashboardController
- AdminTransactionHistoryApiController
- AdminCurrencyApiController
- AdminCurrencyController
- AdminInvestmentController

Auth Controller

- AuthController

Client Additional Feature Controllers

- ClientDashboardController
- ClientGlobalModel

- ClientProfileApiController
- ClientProfileController

Client Financial Controllers

- CurrencyConverterController
- InvestmentController

Site Controllers

- GlobalNavbarControllerAdvice
- HomeController

Config Package Overview

The config package contains the core configuration classes responsible for initializing, securing, and managing the Spring MVC application. These classes configure the database connection, web application setup, Spring MVC framework, and Spring Security integration for the Enomy-Finances system.

---

1. DatabaseConfig.java

Purpose

The DatabaseConfig class configures the database connection and initializes the JDBC template used for database operations throughout the system.

Key Responsibilities

- Configures MySQL database connection
- Creates the DataSource bean
- Initializes JdbcTemplate for DAO operations
- Connects the Spring MVC application to the enomy_finance_system database

Importance in the System

This class acts as the foundation of the database layer and allows DAO classes to execute SQL queries and interact with the database.

2. SecurityConfig.java

Purpose

The SecurityConfig class configures Spring Security for authentication, authorization, login handling, and access control.

Key Responsibilities

- Configures secure authentication using Spring Security
- Implements password encryption using BCrypt
- Defines role-based access control:
    - CLIENT
    - ADMIN
- Configures login and logout behavior

- Protects secured routes such as:
  - /client/**
  - /admin/**
- Integrates custom login success and failure handlers

Importance in the System

This class secures the application by ensuring only authorized users can access protected modules and system features.

3. SecurityWebApplicationInitializer.java

Purpose

The SecurityWebApplicationInitializer class initializes and registers the Spring Security filter chain within the web application.

Key Responsibilities

- Automatically loads Spring Security configuration
- Registers security filters for request protection
- Enables security integration without XML configuration

Importance in the System

This class activates Spring Security during application startup and ensures all requests pass through security validation.

4. WebAppInitializer.java

Purpose

The WebAppInitializer class initializes the Spring MVC web application and replaces the traditional web.xml configuration.

Key Responsibilities

- Registers root configuration classes:
  - DatabaseConfig
  - SecurityConfig
- Registers WebConfig
- Maps the Dispatcher Servlet to /
- Configures multipart file upload settings

Importance in the System

This class acts as the main entry point of the Spring MVC application and initializes the entire web system configuration.

5. WebConfig.java

Purpose

The WebConfig class configures the Spring MVC framework, JSP view resolution, static resources, and multipart file handling.

Key Responsibilities

- Enables Spring MVC configuration
- Configures JSP view resolver:
  - /WEB-INF/views/
- Registers static resource handlers:
  - CSS
  - JavaScript
  - Images
  - Uploaded files
- Configures multipart file upload resolver

Importance in the System

This class manages how frontend pages, static resources, and uploaded files are handled within the web application.

Overall Importance of the Config Package

The config package forms the core infrastructure of the Enomy-Finances system by managing:

- Database connectivity
- Spring MVC initialization
- Web application configuration
- Resource handling
- Authentication and security

Security Package Overview

The security package contains the custom security classes responsible for user authentication, login handling, role-based redirection, and user identity management in the Enomy-Finances system. These classes extend Spring Security to match the specific login and access requirements of the application.

1. CustomAuthenticationFailureHandler.java

Purpose

The CustomAuthenticationFailureHandler class handles failed login attempts and records them in the system.

Key Responsibilities

- Detects unsuccessful login attempts
- Retrieves the entered username/email from the request or session
- Checks whether the user exists in the database
- Records failed login activity with:
    - user ID
    - timestamp
    - failure status
    - reason
    - IP address
    - browser/device details
- Redirects the user back to the login page with an error flag

Importance in the System

This class improves security monitoring by tracking failed login attempts and helps support audit and user activity monitoring.

2. CustomAuthenticationSuccessHandler.java

Purpose

The CustomAuthenticationSuccessHandler class handles successful login events and redirects users based on their role.

Key Responsibilities

- Detects successful authentication
- Retrieves the logged-in user from the database

- Updates the user's last login timestamp
- Saves login activity with:
    - success status
    - login time
    - IP address
    - browser/device details
- Redirects users to the correct dashboard:
    - Admin → /admin/dashboard
    - Client → /client/dashboard

Importance in the System

This class ensures secure role-based navigation after login and maintains accurate login history records.

3. CustomUserDetails.java

Purpose

The CustomUserDetails class extends Spring Security's default User class to include additional user information.

Key Responsibilities

- Stores the authenticated user's full name
- Inherits standard Spring Security user properties:
    - username
    - password
    - enabled status
    - authorities/roles
- Provides a getter for the full name

Importance in the System

This class allows the application to access custom user data during authentication and session handling, especially for personalized UI display.

4. CustomUserDetailsService.java

Purpose

The CustomUserDetailsService class loads user information from the database for Spring Security authentication.

Key Responsibilities

- Retrieves a user by email using UserDao
- Throws an exception if the user is not found
- Builds a CustomUserDetails object using:
    - email
    - password
    - enabled status
    - role
    - full name
- Converts the stored system role into Spring Security authority format:
    - ROLE_ADMIN
    - ROLE_CLIENT

Importance in the System

This class connects the database user records to Spring Security and makes authentication possible using stored account details.

Overall Importance of the Security Package

The security package customizes Spring Security for the Enomy-Finances system by managing:

- login success and failure behavior
- role-based dashboard redirection
- login activity tracking
- secure user identity loading
- personalized authenticated user details

Together, these classes ensure that the authentication process is secure, role-aware, and fully integrated with the database and system workflows.

Model Package Overview

The model package contains the core entity classes of the Enomy-Finances system. These classes represent the main business data and are used to store, retrieve, and manage information throughout the application. The model classes are grouped into three main modules: Currency Conversion, Investment Management, and Enomy-Finances User Management.

1. Conversion Module Models

These model classes manage the currency conversion and transaction processing features of the system.

ConversionFeeRule.java

Purpose

Represents transaction fee rules applied during currency conversion transactions.

Key Responsibilities

- Stores minimum and maximum transaction ranges
- Defines the fee percentage applied to transactions
- Supports dynamic fee calculation during conversion processing

Importance in the System

Allows the system to apply different conversion fee rates based on transaction amount ranges.

ConversionRuleSet.java

Purpose

Represents grouped conversion fee configurations used by the system.

Key Responsibilities

- Stores rule set information
- Manages active/inactive fee configurations
- Organizes conversion fee rules into manageable groups

Importance in the System

Allows administrators to manage and activate specific conversion fee configurations.

CurrencyTransaction.java

Purpose

Represents completed currency conversion transactions within the system.

Key Responsibilities

- Stores transaction details:
  - currencies
  - exchange rate
  - fees
  - final converted amount
- Records transaction status and timestamps
- Links transactions to users and applied fee rules

Importance in the System

Acts as the main financial transaction record for the currency conversion module.

2. Investment Module Models

These model classes manage savings plans, investment calculations, and tax configurations.

InvestmentQuote.java

Purpose

Represents investment calculation records generated by users.

Key Responsibilities

- Stores investment details:
  - plan type
  - monthly investment
  - initial lump sum
- Links investment quotes to selected investment rules
- Saves generated investment calculations

Importance in the System

Allows users to save and review investment projections and financial planning records.

PlanRules.java

Purpose

Represents investment plan configurations and financial rules.

Key Responsibilities

- Defines:
  - return rates
  - fee rates
  - investment limits
  - minimum investment requirements
- Links investment plans to tax settings
- Supports admin-configurable investment rules

Importance in the System

Provides the financial logic and validation rules used during investment calculations.

TaxSettings.java

Purpose

Represents tax configuration settings used in investment calculations.

Key Responsibilities

- Stores tax brackets and tax rates
- Defines tax-free allowances and thresholds
- Supports dynamic tax calculations

Importance in the System

Ensures investment calculations follow configurable tax rules and financial policies.

3. Enomy-Finances User Management Models

These model classes manage user accounts, authentication records, and login monitoring.

User.java

Purpose

Represents registered users within the Enomy-Finances system.

Key Responsibilities

- Stores user account information:
  - full name
  - email
  - password
  - role
- Tracks login timestamps and profile details
- Supports account activation and soft deletion

Importance in the System

Acts as the primary user entity for authentication, authorization, and profile management.

LoginActivity.java

Purpose

Represents login activity records generated during authentication attempts.

Key Responsibilities

- Stores login attempt details:
  - login status
  - timestamps
  - IP address
  - browser/device information
- Records both successful and failed login attempts

Importance in the System

Supports login monitoring, security auditing, and user activity tracking.

Overall Importance of the Model Package

The model package forms the core data structure of the Enomy-Finances system by representing:

- financial transactions
- investment configurations
- tax settings
- user accounts
- authentication activity

These model classes ensure consistent data management and provide the foundation for the DAO, Service, and Controller layers of the application.

Conversion DTO Package Overview

The conversion DTO classes are used to transfer currency conversion data between the controller, service, DAO, and frontend layers of the Enomy-Finances system. They help organize request data, calculation results, admin records, API responses, and transaction summaries for the currency conversion module.

---

1. User Conversion DTOs

These DTOs support the client-side currency conversion process.

CurrencyConversionRequestDTO.java

Purpose

Carries the user's conversion input data from the frontend to the backend.

Key Responsibilities

- Stores:
  - transaction type
  - base currency
  - target currency
  - amount

Importance in the System

Used when the client submits a currency conversion request.

CurrencyConversionResponseDTO.java

Purpose

Stores the processed conversion result returned by the system after calculation.

Key Responsibilities

- Stores:
  - exchange rate used
  - converted amount
  - fee rate
  - fee value
  - final amount

    - result label
    - validation status
    - response message

Importance in the System

Used to display detailed conversion results and validation feedback to the user.

CheckRateResponseDTO.java

Purpose

Represents the response for quick exchange rate checking before confirming a transaction.

Key Responsibilities

- Stores:
    - base currency
    - target currency
    - rate
    - converted amount
    - rate date

Importance in the System

Used for AJAX-based rate checking and previewing conversion values.

TransactionReceiptDTO.java

Purpose

Represents the final transaction receipt details after a conversion is confirmed and saved.

Key Responsibilities

- Stores:
  - transaction number
  - date
  - currencies
  - amounts
  - applied fee
  - final amount
  - status message

Importance in the System

Used to display or return a complete conversion receipt to the client after saving the transaction.

2. Admin Conversion DTOs

These DTOs support the administrator side of the currency conversion module.

AdminCurrencyRateRowDTO.java

Purpose

Represents exchange rate records displayed in the admin currency rate table.

Key Responsibilities

- Stores:
  - base currency
  - target currency
  - exchange rate
  - rate date
  - retrieved timestamp

Importance in the System

Used to present currency rate information in the admin interface.

AdminCurrencyTransactionHistoryRowDTO.java

Purpose

Represents transaction history rows displayed in the admin currency transaction history table.

Key Responsibilities

- Stores:
  - transaction number
  - user ID
  - user name
  - transaction type
  - currencies
  - amounts
  - status
  - creation date

Importance in the System

Used by administrators to monitor and review all currency transactions across the system.

3. Conversion Rule Management DTOs

These DTOs support the management of conversion rule sets and fee rules.

ConversionFeeRuleDTO.java

Purpose

Represents a single conversion fee rule in a simplified transfer format.

Key Responsibilities

- Stores:
  - minimum amount
  - maximum amount
  - fee rate

Importance in the System

Used when creating, editing, or transferring fee rule configurations.

ConversionRuleSetDTO.java

Purpose

Represents a full conversion rule set together with its fee rule list.

Key Responsibilities

- Stores:
  - rule set name
  - description
  - list of fee rules

Importance in the System

Used for admin-side rule set creation and management.

4. External API DTO

CurrencyRateApiDTO.java

Purpose

Represents exchange rate data retrieved from the external currency API.

Key Responsibilities

- Stores:
  - exchange rate
  - rate date

Importance in the System

Acts as the data container for external API responses before rates are processed by the application.

Overall Importance of the Conversion DTO Package

The conversion DTO classes help separate input, output, API data, admin table data, and transaction summaries from the core model classes. This improves:

- clean data transfer
- modular system design
- easier frontend/backend integration
- better separation of concerns in the currency conversion module

Investment DTO Package Overview

The investment DTO classes are used to transfer investment-related data between the controller, service, DAO, and frontend layers of the Enomy-Finances system. These DTOs support user input, investment result generation, plan display, tax history management, and admin-side investment monitoring.

---

1. User Investment DTOs

These DTOs support the client-side investment calculation process.

InvestmentRequestDTO.java

Purpose

Carries the user's investment input data from the frontend to the backend.

Key Responsibilities

- Stores:
  - plan type
  - initial lump sum
  - monthly investment

Importance in the System

Used when the client submits an investment calculation request.

InvestmentResponseDTO.java

Purpose

Stores the complete investment calculation results returned by the system.

Key Responsibilities

- Stores the selected plan type
- Holds result summaries for:

    - 1 year
    - 5 years
    - 10 years

Importance in the System

Used to return structured investment projection results to the frontend.

YearlyInvestmentResultDTO.java

Purpose

Represents the detailed financial result for a specific investment period.

Key Responsibilities

- Stores:
    - years
    - total invested amount
    - minimum and maximum returns
    - minimum and maximum profit
    - minimum and maximum tax
    - monthly fee
    - total fee

Importance in the System

Used to show detailed financial outcomes for each time period in the investment module.

PlanDetailsDTO.java

Purpose

Represents the displayed details of an investment plan for the user interface.

Key Responsibilities

- Stores:
  - plan title
  - maximum yearly investment
  - minimum monthly investment
  - minimum initial investment
  - predicted returns
  - estimated tax
  - monthly group fees

Importance in the System

Used to present readable plan information to users before calculation.

2. Admin Investment DTOs

These DTOs support administrator-side monitoring and management of investment data.

AdminInvestmentQuoteHistoryRowDTO.java

Purpose

Represents investment quote history rows displayed in the admin interface.

Key Responsibilities

- Stores:
  - quote ID
  - user ID
  - user name
  - plan type
  - initial lump sum
  - monthly investment

    - plan rule ID
    - creation date

Importance in the System

Used by administrators to view and monitor saved investment quotes across the system.

TaxSetHistoryDTO.java

Purpose

Represents grouped tax setting history for admin-side tax configuration management.

Key Responsibilities

- Stores:
    - tax set ID
    - creation date
    - active status
- Holds grouped tax configurations:
    - none tax
    - flat tax
    - progressive tax

Importance in the System

Used to organize and display tax configuration history for administrator management.

Overall Importance of the Investment DTO Package

The investment DTO classes separate user input, calculation results, plan information, and admin monitoring data from the core model classes. This improves:

- clean data transfer
- structured investment result handling
- clearer admin reporting
- better separation of concerns in the investment module

Enomy-Finances User DTO Package Overview

The Enomy-Finances User DTO classes are used to transfer profile management, authentication activity, and account update data between the frontend, controller, service, and DAO layers of the system. These DTOs support user profile operations, password management, login monitoring, and account customization features.

1. Profile Management DTOs

These DTOs support user profile updates and profile page data handling.

ClientProfilePageDTO.java

Purpose

Represents the complete profile page data displayed to the user.

Key Responsibilities

- Stores:
  - user information
  - login activity history
  - failed login statistics
  - previous successful login
  - latest successful and failed login records

Importance in the System

Used to generate and display the complete client profile dashboard and security activity information.

ProfileInfoUpdateDTO.java

Purpose

Represents profile information updates submitted by the user.

Key Responsibilities

- Stores updated full name information

Importance in the System

Used when users update their profile details.

ProfileAvatarUpdateDTO.java

Purpose

Represents profile avatar or image update requests.

Key Responsibilities

- Stores the updated profile image path

Importance in the System

Used for profile photo customization and avatar updates.

2. Security & Password DTOs

These DTOs support account security and password management operations.

ChangePasswordDTO.java

Purpose

Represents password change requests submitted by authenticated users.

Key Responsibilities

- Stores:
    - current password
    - new password
    - password confirmation

Importance in the System

Used to validate and process secure password update operations.

3. Login Activity & Filtering DTOs

These DTOs support login activity monitoring and filtering functionality.

LoginActivityFilterDTO.java

Purpose

Represents filter criteria for searching login activity records.

Key Responsibilities

- Stores:
  - start date
  - end date

Importance in the System

Used to filter login history records within specific date ranges.

Overall Importance of the Enomy-Finances User DTO Package

The user DTO classes separate profile management, password handling, and login activity data from the core user model. This improves:

- clean frontend/backend communication
- secure profile update handling
- organized login monitoring
- modular account management features

These DTOs support the user account management and security functionalities of the Enomy-Finances system.

Conversion DAO and DAO Implementation Overview

The conversion DAO and DAO implementation classes are responsible for handling database operations for the currency conversion module of the Enomy-Finances system. They provide structured access to conversion fee rules, conversion rule sets, and currency transaction records using JDBC and MySQL.

1. Conversion Fee Rule DAO

ConversionFeeRuleDao.java

Purpose

Defines the database operations related to conversion fee rules.

Key Responsibilities

- Save a conversion fee rule
- Retrieve fee rules by rule set ID
- Find the matching fee rule for a transaction amount
- Retrieve all fee rules in sorted order

Importance in the System

This interface defines how fee rule data is accessed and managed for dynamic conversion fee calculation.

ConversionFeeRuleDaoImpl.java

Purpose

Implements the database logic for managing conversion fee rules using JdbcTemplate.

Key Responsibilities

- Inserts fee rules into the conversion_fee_rule table
- Retrieves fee rules belonging to a specific rule set
- Finds the correct fee rule based on transaction amount range
- Retrieves all fee rules ordered by rule set and minimum amount

Importance in the System

This implementation enables the system to apply the correct fee percentage during currency conversion transactions.

2. Conversion Rule Set DAO

ConversionRuleSetDao.java

Purpose

Defines the database operations related to conversion rule sets.

Key Responsibilities

- Find the currently active rule set
- Save a new conversion rule set
- Deactivate all existing rule sets
- Find the maximum rule set ID
- Find a rule set by ID
- Activate a selected rule set
- Retrieve all rule sets ordered by creation date

Importance in the System

This interface manages the fee rule groupings used in the currency conversion module.

ConversionRuleSetDaoImpl.java

Purpose

Implements the database logic for managing conversion rule sets using JdbcTemplate.

Key Responsibilities

- Retrieves the active conversion rule set
- Inserts new rule sets into the conversion_rule_set table
- Deactivates all existing rule sets before activating another
- Retrieves rule sets by ID or creation date order

- Activates a selected rule set

Importance in the System

This implementation allows the administrator to manage which conversion fee rule set is currently active in the system.

3. Currency Transaction DAO

CurrencyTransactionDao.java

Purpose

Defines the database operations related to currency conversion transactions.

Key Responsibilities

- Save a currency transaction
- Retrieve transactions by user ID
- Find a transaction by transaction number and user ID
- Count transactions for a user
- Retrieve filtered transaction records
- Retrieve admin transaction history

Importance in the System

This interface manages how transaction records are stored, searched, and monitored in both client and admin modules.

CurrencyTransactionDaoImpl.java

Purpose

Implements the database logic for managing currency transaction records using JdbcTemplate.

Key Responsibilities

- Inserts completed currency transactions into the currency_transaction table
- Retrieves user-specific transaction history
- Retrieves a single transaction receipt using transaction number and user ID
- Counts user transactions
- Supports filtered transaction history queries
- Supports admin transaction history with advanced search options:
    - exact user ID search
    - transaction number search
    - user name search

Importance in the System

This implementation is the core persistence layer for the currency conversion module, allowing both clients and administrators to view, filter, and manage transaction records.

Overall Importance of the Conversion DAO Layer

The conversion DAO layer separates database access logic from the service and controller layers. It manages:

- conversion fee rules
- rule set activation
- transaction storage
- filtered transaction retrieval
- admin transaction monitoring

This improves:

- clean architecture

- maintainable database operations
- modular system design
- efficient transaction data handling

Investment DAO and DAO Implementation Overview

The investment DAO and DAO implementation classes are responsible for handling database operations for the savings and investment module of the Enomy-Finances system. They manage investment quote records, investment plan rules, and tax settings using JDBC and MySQL.

---

1. Investment Quote DAO

InvestmentQuoteDao.java

Purpose

Defines the database operations related to saved investment quotes.

Key Responsibilities

- Save an investment quote
- Count saved quotes by user
- Retrieve quotes by user ID
- Retrieve a specific quote by quote ID and user ID
- Retrieve admin investment quote history

Importance in the System

This interface manages how user investment quote records are stored and retrieved in both client and admin modules.

InvestmentQuoteDaoImpl.java

Purpose

Implements the database logic for managing saved investment quotes using JdbcTemplate.

Key Responsibilities

- Inserts saved quotes into the investment_quotes table
- Retrieves user-specific saved quote history
- Retrieves a single saved quote for detailed viewing
- Counts total saved quotes of a user
- Supports admin investment quote history with filters:
    - plan type
    - date range
    - user ID or user name search

Importance in the System

This implementation is the persistence layer for saving and managing user investment quote records.

2. Plan Rules DAO

PlanRulesDao.java

Purpose

Defines the database operations related to investment plan configurations.

Key Responsibilities

- Find the active plan rule by plan type
- Find a plan rule by ID
- Retrieve the active plan set
- Retrieve all plan rules ordered for history
- Generate the next plan set ID
- Deactivate all plan rules
- Insert a new plan rule
- Retrieve plan rules by plan set ID

Importance in the System

This interface manages the configurable investment plan rules used in the savings and investment module.

PlanRulesDaoImpl.java

Purpose

Implements the database logic for managing investment plan rules and their related tax settings using JdbcTemplate.

Key Responsibilities

- Retrieves active plan rules by plan type
- Retrieves plan rules by ID
- Retrieves the active set of plan rules

- Retrieves full plan rule history for admin use
- Generates the next grouped plan_set_id
- Deactivates all plan rules before activating a new set
- Inserts new plan rule records
- Retrieves grouped plan rules by plan set ID
- Joins plan_rules with tax_settings for complete financial rule mapping

Importance in the System

This implementation provides the financial rule configuration used by the investment calculator and supports administrator control over plan versioning.

3. Tax Settings DAO

TaxSettingsDao.java

Purpose

Defines the database operations related to tax settings used in investment calculations.

Key Responsibilities

- Find a tax setting by ID
- Find the active tax setting
- Retrieve all tax settings ordered for history
- Deactivate all tax settings
- Activate tax settings by ID or set ID
- Insert new tax settings
- Retrieve the active tax set

- Retrieve tax settings by tax set ID
- Generate the next tax set ID

Importance in the System

This interface manages the configurable tax rules that are linked to investment plan calculations.

TaxSettingsDaoImpl.java

Purpose

Implements the database logic for managing grouped tax settings using JdbcTemplate.

Key Responsibilities

- Maps tax_settings rows into TaxSettings objects
- Retrieves individual or active tax settings
- Retrieves all tax settings ordered by tax set history
- Deactivates all tax settings before activating another set
- Inserts new tax setting rows and returns generated IDs
- Retrieves active grouped tax sets
- Retrieves tax settings by tax set ID
- Generates the next tax_set_id
- Activates all tax settings belonging to one grouped set

Importance in the System

This implementation supports dynamic tax rule management and allows tax settings to be versioned and grouped for administrator control.

Overall Importance of the Investment DAO Layer

The investment DAO layer separates database access logic from the service and controller layers. It manages:

- saved investment quotes
- investment plan configurations
- grouped plan versions
- tax settings and tax history
- admin investment quote monitoring

This improves:

- clean architecture
- structured financial rule management
- maintainable database operations
- scalable investment module design

Enomy-Finances User DAO and DAO Implementation Overview

The Enomy-Finances User DAO and DAO implementation classes are responsible for handling database operations related to user accounts, authentication records, profile management, and login activity monitoring. These classes use JDBC and MySQL to manage user-related data throughout the system.

---

1. Login Activity DAO

LoginActivityDao.java

Purpose

Defines the database operations related to user login activity records.

Key Responsibilities

- Save login activity records
- Retrieve all login activities of a user
- Filter login activities by date range
- Retrieve failed login attempts

- Count failed logins:
    - today
    - this month
- Retrieve:
    - last failed login
    - last successful login
    - previous successful login

Importance in the System

This interface supports login monitoring, security tracking, and profile activity management.

LoginActivityDaoImpl.java

Purpose

Implements the database logic for managing login activity records using JdbcTemplate.

Key Responsibilities

- Inserts login activity into the user_login_activity table
- Retrieves complete login history for a user
- Filters login records by date range
- Retrieves failed login attempts only
- Counts failed logins for security statistics
- Retrieves recent successful and failed login records
- Maps database records into LoginActivity objects

Importance in the System

This implementation provides the persistence layer for login security monitoring and profile activity tracking.

2. User DAO

UserDao.java

Purpose

Defines the database operations related to user account management.

Key Responsibilities

- Save a new user account
- Find users by:
    - email
    - ID
    - full name
- Update:
    - full name
    - password
    - profile image
    - last login timestamp
- Retrieve password update timestamp
- Clear profile image
- Soft delete user accounts

Importance in the System

This interface manages user authentication, profile updates, and account lifecycle operations.

UserDaoImpl.java

Purpose

Implements the database logic for managing user accounts using JdbcTemplate.

Key Responsibilities

- Inserts new users into the users table
- Retrieves user accounts by email, ID, or full name
- Updates:
    - profile information
    - password hashes
    - profile image paths
    - login timestamps
- Supports soft deletion by disabling accounts
- Maps database records into User objects
- Filters out deleted accounts during retrieval

Importance in the System

This implementation acts as the core persistence layer for authentication, user management, and profile operations.

Overall Importance of the Enomy-Finances User DAO Layer

The Enomy-Finances User DAO layer separates user-related database access logic from the service and controller layers. It manages:

- user authentication records
- login activity monitoring
- profile updates
- password management
- account lifecycle operations

This improves:

- clean architecture
- secure user management
- organized profile handling
- maintainable authentication operations

Admin Service Layer Overview

The admin service classes contain the business logic for administrator-side features in the Enomy-Finances system. They act as the middle layer between the admin controllers and the DAO layer, handling rule activation, history retrieval, validation, filtering, and grouped financial configuration logic.

---

1. Admin Currency Service

AdminCurrencyService.java

Purpose

Defines the business operations for administrator-side currency management.

Key Responsibilities

- Get the active conversion rule set
- Get active conversion fee rules
- Preview the next rule set ID
- Create and activate a new fee rule set
- Activate an existing rule set
- Retrieve rule set and fee rule history
- Retrieve filtered currency rates

- Retrieve filtered currency transactions

Importance in the System

This interface defines the admin-side business logic for managing currency fee rules, exchange rate viewing, and transaction monitoring.

AdminCurrencyServiceImpl.java

Purpose

Implements the administrator business logic for the currency conversion module.

Key Responsibilities

- Retrieves the currently active fee rule set and fee brackets
- Calculates derived minimum and maximum transaction amounts from rule brackets
- Creates and activates new fee rule sets
- Validates fee brackets for:
    - complete inputs
    - non-overlapping ranges
    - continuous bracket flow
    - valid fee values
- Retrieves all historical rule sets and fee rules
- Retrieves filtered historical exchange rates through the currency API service
- Retrieves filtered currency transaction records for admin monitoring

Importance in the System

This class is the core business logic layer for admin currency management, ensuring that fee rules are valid, activation is controlled, and transaction/rate history is properly handled.

2. Admin Investment Service

AdminInvestmentService.java

Purpose

Defines the business operations for administrator-side investment plan and tax management.

Key Responsibilities

- Retrieve active plan rule sets
- Retrieve active grouped tax settings
- Retrieve plan and tax history
- Retrieve one plan set by plan set ID
- Retrieve one tax set by tax set ID
- Create new plan rule sets
- Create and activate grouped tax settings
- Clone active plan rules when tax settings change
- Activate existing tax sets with cloned plan rules

Importance in the System

This interface defines the admin-side financial configuration logic for the investment module.

AdminInvestmentServiceImpl.java

Purpose

Implements the administrator business logic for investment plan rules and grouped tax settings.

Key Responsibilities

- Retrieves active plan rule sets and active tax sets
- Retrieves full plan history and grouped tax history
- Groups tax rows into TaxSetHistoryDTO
- Creates new plan rule sets using active or newly created tax settings
- Creates and activates new grouped tax sets
- Activates existing tax sets
- Clones current active plan rules when tax settings are changed
- Maps investment plan types to the correct tax types:
    - BASIC_SAVINGS → NONE
    - SAVINGS_PLUS → FLAT
    - MANAGED_STOCKS → PROGRESSIVE

Importance in the System

This class handles the complex admin-side business logic for plan versioning, grouped tax configuration, and dynamic financial rule management in the investment module.

3. Admin Transaction History Service

AdminTransactionHistoryService.java

Purpose

Defines the business operations for administrator-side transaction history monitoring.

Key Responsibilities

- Retrieve filtered currency transaction history
- Retrieve filtered investment quote history

Importance in the System

This interface defines the admin-side history monitoring logic for both major financial modules.

AdminTransactionHistoryServiceImpl.java

Purpose

Implements the business logic for administrator-side viewing of currency transactions and investment quote history.

Key Responsibilities

- Retrieves filtered currency transaction history through the conversion DAO
- Retrieves filtered investment quote history through the investment DAO
- Normalizes blank filter values before sending them to the DAO layer

Importance in the System

This class provides a clean service layer for admin history filtering and reporting across both conversion and investment modules.

Overall Importance of the Admin Service Layer

The admin service layer is responsible for applying business rules and managing administrator workflows in the Enomy-Finances system. It handles:

- conversion rule creation and validation
- rule activation and history
- investment plan and tax configuration
- grouped tax set processing
- admin transaction monitoring
- filtered reporting and historical record retrieval

This improves:

- separation of concerns
- reusable business logic
- clean controller-to-database flow
- maintainable administrator operations

Client Service Layer Overview

The client service classes contain the business logic for client-side operations in the Enomy-Finances system. These services process user requests, validate inputs, perform financial calculations, integrate with external APIs, manage profile operations, and coordinate data flow between the controller and DAO layers.

---

1. Client Profile Service

ClientProfileService.java

Purpose

Defines the business operations related to client profile management and account security.

Key Responsibilities

- Retrieve profile page data
- Retrieve user account information
- Update:
  - full name
  - password
  - profile image
- Retrieve login activity records
- Retrieve failed login activity records
- Soft delete user accounts
- Save uploaded profile images

Importance in the System

This interface manages profile customization, account security, and login activity monitoring for clients.

ClientProfileServiceImpl.java

Purpose

Implements the business logic for client profile management and security features.

Key Responsibilities

- Builds complete profile page data using:
  - user records
  - login activity statistics
  - successful and failed login history
- Updates user profile information
- Validates current password before updating to a new password
- Encrypts new passwords using PasswordEncoder
- Retrieves filtered login activity records

- Handles profile image upload:
  - generates unique file names
  - creates upload directories
  - stores uploaded images
  - updates image paths in the database
- Supports soft account deletion

Importance in the System

This class manages account security, profile customization, and activity tracking for the client profile module.

2. Currency API Service

CurrencyApiService.java

Purpose

Defines the operations for retrieving live and historical exchange rates from an external currency API.

Key Responsibilities

- Retrieve current exchange rates
- Retrieve exchange rates with retrieval date
- Retrieve historical exchange rates
- Retrieve historical rates with dates

Importance in the System

This interface provides the integration layer between the Enomy-Finances system and the external currency exchange API.

CurrencyApiServiceImpl.java

Purpose

Implements the external API integration for retrieving exchange rates.

Key Responsibilities

- Connects to the Frankfurter Currency API
- Retrieves:
  - live exchange rates
  - historical exchange rates
- Parses JSON API responses using Jackson ObjectMapper
- Builds CurrencyRateApiDTO objects
- Handles API connection and response validation
- Supports date-based historical rate retrieval

Importance in the System

This class enables real-time and historical currency conversion functionality in the application.

3. Currency Converter Service

CurrencyConverterService.java

Purpose

Defines the business operations for the client-side currency conversion module.

Key Responsibilities

- Retrieve the active conversion rule set
- Check exchange rates
- Calculate currency conversions
- Confirm and save transactions
- Retrieve transaction history

Importance in the System

This interface manages the financial processing logic of the currency conversion module.

CurrencyConverterServiceImpl.java

Purpose

Implements the business logic for currency conversion, fee calculation, validation, and transaction processing.

Key Responsibilities

- Retrieves active conversion fee rules
- Validates:
  - currencies
  - transaction type
  - transaction amount
- Retrieves live exchange rates from the API service
- Calculates:
  - converted amount
  - fee values
  - payable/received amount
- Applies different logic for:
  - BUY transactions
  - SELL transactions
- Generates transaction numbers
- Saves completed transactions
- Builds transaction receipts
- Retrieves transaction history

Importance in the System

This class is the core financial processing engine for the currency conversion module.

4. Investment Service

InvestmentService.java

Purpose

Defines the business operations for the savings and investment module.

Key Responsibilities

- Calculate investment projections
- Save investment quotes
- Count saved quotes
- Retrieve saved quotes
- Retrieve saved quote details
- Retrieve active investment plan details
- Retrieve all active plan information

Importance in the System

This interface manages the financial projection and investment calculation logic of the system.

InvestmentServiceImpl.java

Purpose

Implements the business logic for investment calculations, financial projections, validation, and quote management.

Key Responsibilities

- Validates investment requests and financial limits
- Retrieves active plan rules
- Calculates:
  - projected returns
  - projected profits
  - taxes
  - monthly and total fees
- Generates projections for:
  - 1 year
  - 5 years
  - 10 years
- Applies:
  - flat tax rules
  - progressive tax rules
  - tax-free allowances
- Saves investment quotes
- Retrieves saved quote details
- Builds plan detail summaries for frontend display
- Formats percentages, tax labels, and financial outputs

Importance in the System

This class is the core financial calculation engine for the investment module and handles complex projection and taxation logic.

Overall Importance of the Client Service Layer

The client service layer contains the main business processing logic of the Enomy-Finances system. It manages:

- user profile operations
- password security

- login activity tracking
- external currency API integration
- currency conversion calculations
- transaction processing
- investment projections
- tax and fee calculations
- saved financial records

This improves:

- separation of concerns
- reusable business logic
- clean controller-to-database communication
- scalable financial processing architecture

Admin Controller Layer Overview

The admin controllers handle administrator-side page routing, request handling, and interaction between the frontend and the admin service layer. They are responsible for loading admin pages, preparing model data for JSP views, and exposing AJAX/API endpoints for dynamic admin operations.

---

1. AdminDashboardController.java

Purpose

The AdminDashboardController manages basic administrator page navigation for the dashboard and transaction history views.

Key Responsibilities

- Loads the admin dashboard page
- Loads the admin transaction history page
- Retrieves authenticated admin details from CustomUserDetails

- Sends topbar data to the view:
  - full name
  - logged-in email
  - active page indicator

Importance in the System

This controller provides the entry point for administrator navigation and ensures the correct dashboard context is shown in the admin interface.

2. AdminTransactionHistoryApiController.java

Purpose

The AdminTransactionHistoryApiController exposes REST API endpoints for filtering and retrieving admin transaction history records.

Key Responsibilities

- Handles AJAX filtering for:
  - currency transaction history
  - investment quote history
- Accepts filter request data using request body objects
- Calls AdminTransactionHistoryService
- Returns JSON responses containing:
  - success flag
  - filtered result lists
  - error messages when needed

Importance in the System

This controller powers the dynamic admin history tables and supports filtered monitoring of financial records across the system.

3. AdminCurrencyApiController.java

Purpose

The AdminCurrencyApiController manages AJAX and REST operations for the administrator currency management module.

Key Responsibilities

- Retrieves the active conversion rule set and active fee rules
- Creates and activates new conversion rule sets
- Retrieves conversion rule history
- Retrieves detailed fee bracket information for a selected rule set
- Activates an existing conversion rule set
- Filters currency rate history
- Filters currency transaction history
- Returns structured JSON responses for frontend updates

Importance in the System

This controller is the main API layer for dynamic admin currency operations, including fee rule management, rate viewing, and transaction monitoring.

4. AdminCurrencyController.java

Purpose

The AdminCurrencyController manages page loading for the admin currency management interface.

Key Responsibilities

- Loads the admin/admin-currency page
- Prepares common admin page data
- Sets page state values such as:
    - active page
    - active section

    - active navigation tab
- Loads initial currency management data into the model:
    - active rule set
    - active fee rules
    - min/max transaction range
    - rule history
    - fee rule modal data
    - transaction history

Importance in the System

This controller prepares the admin currency JSP page and ensures all required initial data is available before rendering.

5. AdminInvestmentController.java

Purpose

The AdminInvestmentController manages page routing and form processing for the admin investment management module.

Key Responsibilities

- Loads the admin investment page and its sections:
    - active rules
    - plan rules
    - tax settings
    - history
- Handles form submissions for:
    - creating tax settings
    - activating tax settings
    - creating plan rules using active tax
    - creating plan rules with new tax settings
- Prepares shared page data:
    - topbar details

    - active plan set
    - active tax set
    - plan history
    - tax history
- Builds grouped form objects from submitted request data
- Restores wizard draft data after failed submissions

Importance in the System

This controller is the main page and workflow controller for administrator investment configuration, tax management, and historical financial rule tracking.

Overall Importance of the Admin Controller Layer

The admin controller layer is responsible for:

- administrator page navigation
- loading JSP views
- handling admin form submissions
- exposing AJAX endpoints
- coordinating model data for admin modules

It connects the admin interface to the service layer and enables management of:

- dashboards
- transaction history
- conversion rules
- exchange rate records
- investment plans
- tax settings

Auth Controller Layer Overview

The auth controller handles user authentication-related page navigation and account registration processes in the Enomy-Finances system. It manages login page access, signup validation, account creation, and secure password handling before users enter the secured system modules.

---

1. AuthController.java

Purpose

The AuthController manages authentication-related routes such as login and signup page handling, user registration validation, and account creation.

Key Responsibilities

- Loads the login page
- Loads the signup page
- Processes new user registration
- Validates:
  - password confirmation
  - duplicate email accounts
  - duplicate usernames
- Encrypts passwords using PasswordEncoder and BCrypt
- Creates new client accounts
- Saves registered users into the database
- Returns success and error messages to the frontend

Importance in the System

This controller acts as the entry point of the Enomy-Finances system by managing secure user registration and authentication page navigation before users access protected financial modules.

Overall Importance of the Auth Controller Layer

The auth controller layer is responsible for:

- authentication page routing
- account registration processing
- credential validation
- secure password encryption
- user onboarding

It ensures that:

- only valid accounts are created
- passwords are securely stored
- duplicate accounts are prevented
- users can securely access the system after registration

Client Controller – Additional Feature Overview

The Client Additional Feature controllers handle client-side dashboard, profile management, shared client model data, and AJAX-based profile operations. These controllers connect the client JSP pages to the service layer and support personalized user experience, account security, and profile activity monitoring.

1. ClientDashboardController.java

Purpose

The ClientDashboardController loads the main client dashboard after successful login.

Key Responsibilities

- Retrieves authenticated client details
- Loads the logged-in user from the database
- Counts the client's saved investment quotes
- Retrieves active investment plan details
- Sends dashboard data to the JSP view:
    - full name
    - email
    - active page
    - saved quote count
    - active plan information

Importance in the System

This controller prepares the personalized client dashboard and displays summary information needed for user navigation and financial service access.

2. ClientGlobalModel.java

Purpose

The ClientGlobalModel provides shared user data automatically to all client-side controllers and views.

Key Responsibilities

- Uses @ControllerAdvice for client controllers
- Automatically adds the authenticated user to the model
- Automatically adds the user's full name to the model
- Retrieves user information using the logged-in principal email

Importance in the System

This class reduces repeated code by making common user data available across client pages such as dashboard, profile, currency, and investment pages.

3. ClientProfileApiController.java

Purpose

The ClientProfileApiController provides REST API endpoints for dynamic profile updates and login activity filtering.

Key Responsibilities

- Retrieves profile data through AJAX/API
- Updates profile information such as full name
- Updates password with validation rules
- Updates or removes profile photo path
- Retrieves login activity history with date filtering
- Retrieves failed login records
- Handles account soft deletion
- Returns JSON responses with success/error messages

Validation Handled

- Full name length and required checks
- Password confirmation check
- Password strength validation
- Date range validation
- Authentication checks before updates

Importance in the System

This controller supports dynamic profile management without needing full page reloads and improves user account security through validation and activity tracking.

4. ClientProfileController.java

Purpose

The ClientProfileController loads the client profile page and handles uploaded profile images.

Key Responsibilities

- Loads the client/client-profile page
- Retrieves complete profile page data
- Redirects unauthenticated users to login
- Handles profile image upload
- Validates uploaded image type:
    - PNG
    - JPG
    - JPEG
    - WEBP
- Sends upload success or failure response as JSON

Importance in the System

This controller manages the client profile page and supports profile photo upload functionality, improving personalization and user experience.

Overall Importance of Client Additional Feature Controllers

The Client Additional Feature controllers support:

- client dashboard loading
- shared client user data
- profile page rendering
- dynamic profile updates
- password security
- login activity monitoring
- profile image upload
- soft account deletion

Together, these controllers improve the usability, personalization, and security of the client-side system.

Client Controller – Currency Conversion Overview

The currency conversion controller handles all client-side currency exchange workflows including conversion requests, exchange rate checking, transaction confirmation, receipt generation, and transaction history viewing. It coordinates the interaction between the frontend JSP pages and the currency conversion service layer.

---

1. CurrencyConverterController.java

Purpose

The CurrencyConverterController manages the complete client-side currency conversion workflow and navigation.

Key Responsibilities

- Loads the currency converter page
- Handles navigation between:
  - welcome section
  - buy conversion
  - sell conversion
  - transaction history
- Retrieves active conversion rule sets
- Checks live exchange rates
- Processes conversion calculations
- Confirms and saves currency transactions
- Generates transaction receipts
- Retrieves transaction history
- Handles AJAX exchange rate checking
- Loads topbar and authenticated user information

Features Implemented

- BUY and SELL transaction modes
- Dynamic exchange rate checking
- Conversion result display
- Receipt generation
- Transaction history viewing
- Reset and cancel conversion workflow

Importance in the System

This controller acts as the main interaction layer for the currency conversion module and manages the full conversion process from calculation to transaction completion.

Client Controller – Investment Overview

The investment controller handles all client-side savings and investment workflows including investment calculation, quote saving, quote retrieval, investment projection display, and AJAX-based calculation features.

---

2. InvestmentController.java

Purpose

The InvestmentController manages the savings and investment module for client users.

Key Responsibilities

- Loads the investment page
- Retrieves active investment plan details
- Calculates investment projections
- Saves investment quotes
- Displays saved investment quotes
- Displays detailed quote projections
- Handles AJAX-based:
    - investment calculation
    - quote saving
    - quote detail retrieval
- Retrieves saved quote counts
- Loads authenticated user information
- Formats investment result and projection data

Features Implemented

- Investment projection calculation
- Multi-year financial projection:
    - 1 year

    - 5 years
    - 10 years
- Saved investment quote management
- AJAX-based calculator updates
- Quote detail modal display
- Dynamic plan information display

Validation Handled

- Authentication validation
- Investment calculation validation
- Saved quote ownership validation
- Input validation through service layer

Importance in the System

This controller manages the complete client-side investment workflow and provides dynamic financial planning functionality for users.

Overall Importance of Client Financial Controllers

The client financial controllers manage the main financial services of the Enomy-Finances system including:

- currency conversion
- investment projection
- financial transaction processing
- quote management
- receipt generation
- AJAX-based financial operations

Together, these controllers provide the main user-facing financial functionality of the application and connect the frontend interface to the financial business logic layer.

Site Controller Layer Overview

The site controllers manage public-facing pages and shared navigation data across the Enomy-Finances web application. These controllers support landing pages, public information pages, navbar display logic, and the public exchange rate checker.

---

1. GlobalNavbarControllerAdvice.java

Purpose

The GlobalNavbarControllerAdvice provides shared navbar data across the application.

Key Responsibilities

- Checks whether a user is logged in
- Adds global navbar attributes to the model:
  - login status
  - full name
  - role
  - profile image path
- Retrieves logged-in user details using UserDao
- Supports dynamic navbar display for public and authenticated pages

Importance in the System

This class ensures that the navigation bar updates automatically depending on whether the user is logged in or not.

2. HomeController.java

Purpose

The HomeController manages the public pages of the Enomy-Finances system.

Key Responsibilities

- Loads public pages:
  - home page
  - about page
  - landing currency converter page
  - landing investment page
- Sets the active page indicator for navigation
- Loads active conversion rule data for the public converter page
- Provides AJAX exchange rate checking for public users

Importance in the System

This controller acts as the entry point for public users and allows visitors to explore the system before logging in or registering.

Overall Importance of the Site Controller Layer

The site controller layer supports:

- public page routing
- landing page navigation
- dynamic navbar behavior
- public exchange rate checking
- user-aware page display

It helps create a smooth experience for both visitors and logged-in users.